COMPILER DESIGN AND SYSTEM SOFTWARE
[As per Choice Based Credit System (CBCS) scheme]

SEMESTER – VI

Course Code	:	23CS3602	**Credits**	:	04
Hours / Week	:	04 Hours	**Total Hours**	:	39(T)+ 26(P)Hours
L–T–P–J	:	3–0–2–0			

Course Learning Objectives:

This Course will enable students to:

1 **Explain** the basic assembler design and functionality to identify the process of translating assembly language into machine code.

2 **Differentiate** between various types of loaders and identify the purpose and functionality of linkers and loaders in program execution.

3 **Outline** the purpose and function of lexical analysis and syntax analysis in the compilation process.

4 **Analyze** the principles and mechanics of different top down and bottom up parsing methods.

5 **Apply** various local optimizations (e.g., constant folding, dead code elimination) and global optimizations (e.g., loop transformations, data flow analysis) and its role in efficient code generation.

Teaching-Learning Process (General Instructions)

These are sample new pedagogical methods, where teacher can use to accelerate the attainment of the various course outcomes.

1. ***Lecture method*** means it includes not only traditional lecture method, but different *type of teaching methods* may be adopted to develop the course outcomes.
2. ***Interactive Teaching:*** *Adopt the* ***Active learning*** that includes brainstorming, discussing, group work, focused listening, formulating questions, notetaking, annotating, and roleplaying.
3. Show ***Video/animation*** films to explain functioning of various concepts.
4. Encourage ***Collaborative*** (Group Learning) Learning in the class.
5. To make ***Critical thinking***, ask at least three Higher order Thinking questions in the class.
6. Adopt ***Problem Based Learning***, which fosters students' Analytical skills, develop thinking skills such as the ability to evaluate, generalize, and analyse information rather than simply recall it.
7. Show the ***different ways to solve*** the same problem and encourage the students to come up with their own creative ways to solve them.
8. Discuss how every ***concept can be applied to the real world*** - and when that's possible, it helps improve the students' understanding.

UNIT – I	**9 Hours**

INTRODUCTION TO SYSTEM SOFTWARE:

Machine Architecture of SIC and SIC/XE.

(Text Book-1: Chapter 1: 1.1, 1.2, 1.3)

ASSEMBLERS:

Basic assembler functions: A simple assembler, Assembler algorithm and data structures. ***(Text Book-1: Chapter 2: 2.1)*** **MACHINE DEPENDENT ASSEMBLER FEATURES**: Instruction formats and addressing modes – Program relocation. ***(Text Book-1: Chapter 2: 2.2)*** **MACHINE INDEPENDENT ASSEMBLER FEATURES:** Literals, Symbol-defining statements, Expressions, Program blocks . ***(Text Book-1: Chapter 2: 2.3)***	
UNIT – II	**08 Hours**
LOADERS: Basic loader functions: Design of an Absolute Loader, A Simple Bootstrap Loader ***(Text Book-1: Chapter 3: 3.1)*** **MACHINE DEPENDENT LOADER FEATURES**: Relocation, Program Linking, Algorithm and Data Structures for Linking Loader ***(Text Book-1: Chapter 3: 3.2)*** **MACHINE-INDEPENDENT LOADER FEATURES:** Automatic Library Search, Loader Options, Loader design options: Linkage Editors, Dynamic Linking. ***(Text Book-1: Chapter 3: 3.3)***	
UNIT – III	**8 Hours**
INTRODUCTION TO COMPILERS: Language Processors, Structure of compiler, The science of building a compiler, Applications of compiler technology. ***(Text Book-2: Chapter 1: 1.1, 1.2, 1.4,1.5)*** **LEXICAL AND SYNTAX ANALYSIS**: Role of lexical Analyzer, Specification of Tokens, Lexical Analyzer generator Lex. ***(Text Book-2: Chapter 3: 3.1)*** **SYNTAX ANALYSIS I**: Role of Parser, Syntax error handling, Error recovery strategies, Writing a grammar: Lexical vs Syntactic Analysis, Eliminating ambiguity, Left recursion, Left factoring. ***(Text Book-2: Chapter 4: 4.1, 4.3)***	
UNIT – IV	**08 Hours**
TOP DOWN PARSING: Recursive Descent Parsing, First and follow, LL (1) ***(Text Book-2: Chapter 4: 4.4)*** **BOTTOM UP PARSING:** Shift Reduce Parsing, Introduction to LR parsing Simple LR: Why LR Parsers, Items and LR0 Automaton, The LR Parsing Algorithm. ***(Text Book-2: Chapter 4: 4.5, 4.6)***	

SYNTAX-DIRECTED TRANSLATION: Syntax-Directed Definitions: Inherited and Synthesized Attributes. ***(Text Book-2: Chapter 5: 5.1)*** **EVALUATION ORDERS FOR SDDS:** Dependency graphs, Ordering the evaluation of Attributes, S-Attributed Definition, L-Attributed Definition. ***(Text Book-2: Chapter 5:5.2)*** **APPLICATION:** Construction of Syntax Trees. ***(Text Book-2: Chapter 5: 5.3)***

UNIT – V	**06 Hours**
Three Address Code: Addresses and Instructions, Quadruples, Triples, indirect triples. ***(Text Book-2: Chapter 6: 6.2)*** **CODE GENERATION:** Issues in the design of code generator ***(Text Book-2: Chapter 8: 8.1)*** **Basic Blocks:** Optimization of Basic Blocks, The Code Generation Algorithm, Peephole optimization. ***(Text Book-2: Chapter 8:8.4,8.5 8.6,8.7)*** **MACHINE INDEPENDENT OPTIMIZATION:** The Principal Sources of Optimization ***(Text Book-2: Chapter 9: 9.1)***	

Course Outcome	**Description**	**Bloom's Taxonomy Level**
At the end of the course the student will be able to:		
1	Identify the data structures, algorithm, machine dependent Assembler features and build the object code for Simplified Instructional Computer program.	L3
2	Infer how linker and loader builds an executable program from an object module generated by assembler	L4
3	Interpret the major phases of compilation and to apply the knowledge of Lex tool & YACC tool to build the appropriate parsing application	L3
4	Compare and Contrast various top down and bottom-up parsing techniques to analyze grammatical structures involved in compiler construction and use formal attributed grammars for specifying the syntax and semantics of	L4

	programming languages	
5	Select various optimization techniques used for dataflow analysis and build machine code from the source code of a novel language.	L3

Table: Mapping Levels of COs to POs / PSOs

COs	Program Outcomes (POs)												PSOs	
	1	2	3	4	5	6	7	8	9	10	11	12	1	2
CO1	2		2									2	1	2
CO2				2								1	1	
CO3	2		2		2							2	2	2
CO4	2	3	2	2								2	3	2
CO5	2	3	1	2								2	2	2

3: Substantial (High) **2: Moderate (Medium)** **1: Poor (Low)**

TEXT BOOKS:

1. Leland L. Beck, "System Software – An Introduction to Systems Programming", 3rd Edition, Pearson Education Asia, 2006.
2. Alfred V Aho, Monica S. Lam, Ravi Sethi and Jeffrey D Ullman, Compilers – Principles, Techniques and Tools", 2nd Edition, Pearson Education, 2007.

REFERENCE BOOKS:

1. V. Raghavan, Principles of Compiler Design||, Tata McGraw Hill Education Publishers, 2010.
2. D.M.Dhamdhere, Systems Programming and operating systems, Second Revised edition, Tata McGraw Hill.

E-Resources:

1. https://onlinecourses.nptel.ac.in/noc21_cs07/preview
2. https://www.cs.princeton.edu/courses/archive/spring20/cos320
3. https://ocw.mit.edu/courses/6-004-computation-structures-spring-2017/pages/c11/

Activity Based Learning (Suggested Activities in Class)

a. Using Software Simulators for Parsing techniques.

a. Use interactive tools like online compiler visualizers such as LLVM IR viewer, Godbolt Compiler Explorer. Perform live demonstrations of code compilation. Encourage students to experiment with their code and observe the generated output at each stage.

COMPILER DESIGN AND SYSTEM SOFTWARE LABORATORY

Total Contact Hours: 26

Following are experiments to be carried out using lex and yacc.

1a. Program to count the number of characters, words, spaces and lines in a given input file.
1b. Program to recognize and count the number of identifiers in a file.
2a. Program to count the numbers of comment lines in a given C program. Also eliminate them and copy the resulting program into separate file.
2b. Program to recognize whether a given sentence is simple or compound.
3a. Program to count no of: i.+ve and –ve integers ii. +ve and –ve fractions
3b. Program to count the number of "scanf" and "printf" statements in a C program. Replace them with "readf" and "writef" statements respectively.
4.Program to evaluate arithmetic expression involving operators +,-,*,/
5. Program to recognize a valid variable which starts with a letter, followed by any number of letters or digits.
6. Program to recognize the strings using the grammar ($a_n b_n$;n>=0)
Following are experiments to be carried out using C language.
7. C Program to implement Pass1 of Assembler
8. C Program to implement Absolute Loader
9. C program to find the FIRST in context free grammar.
10.C Program to implement Shift Reduce Parser for the given grammar E →E+E E→E*E E→(E) E → id

OPEN ENDED EXPERIMENTS
1.Write a LR parser program in C. Define the data structure for the parsing table in such a way that it can be initialized easily (manually) for a given grammar. Take a simple grammar, eg., expression grammar, compute the parsing table entries by hand using the steps discussed in the class, and initialize the table in your program with these values. Try to parse input expressions scanned by a lexical analyzer (which can be easily created using flex). The output of the parser should be SUCCESS or FAILURE depending on the input. In case of FAILURE the parser should indicate the incorrect token in the input.
12. Implement a C program to generate three address code for the given expression z = a + b + c – d

13.Write a C program to perform the recursive descent parsing for the given grammar S->AA A->aB/ϵ B->b
14. Write a program to illustrate the loop based code optimization technique.